Digital Signal Processing Exercise Sheet 1

The ECU Sees Only Signals

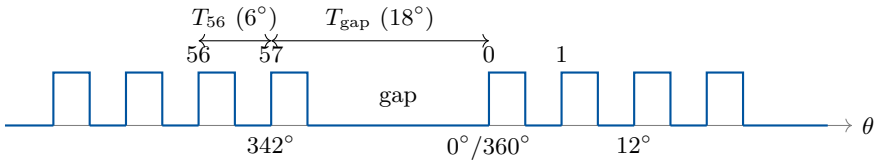
Script: Chapter 1 (The ECU Sees Only Signals) **with solutions**

Learning goals

- Classify the engine sensor signals (continuous/discrete time, analog/binary amplitude, deterministic/stochastic, periodic/aperiodic) and know their characteristic time scales.
- *Bonus (optional):* describe sensor signals quantitatively: mean, RMS, extreme values, autocorrelation as a numerical test for periodicity.
- Extract a first piece of information from a raw signal: crank angle and engine speed from the 60-2 tooth timing, including the effect of the timer resolution.
- Synchronise crank and cam signal to determine the 720° phase of the engine cycle (cylinder identification).

Background: the 60-2 trigger wheel

The crank wheel of the virtual engine has 60 tooth positions (6° each), two of which are missing. The Hall sensor delivers a binary signal; the rising edge of tooth $k = 0, \ldots, 57$ occurs at $\theta = 6k°$ (modulo 360°), the reference mark is the first rising edge after the gap.

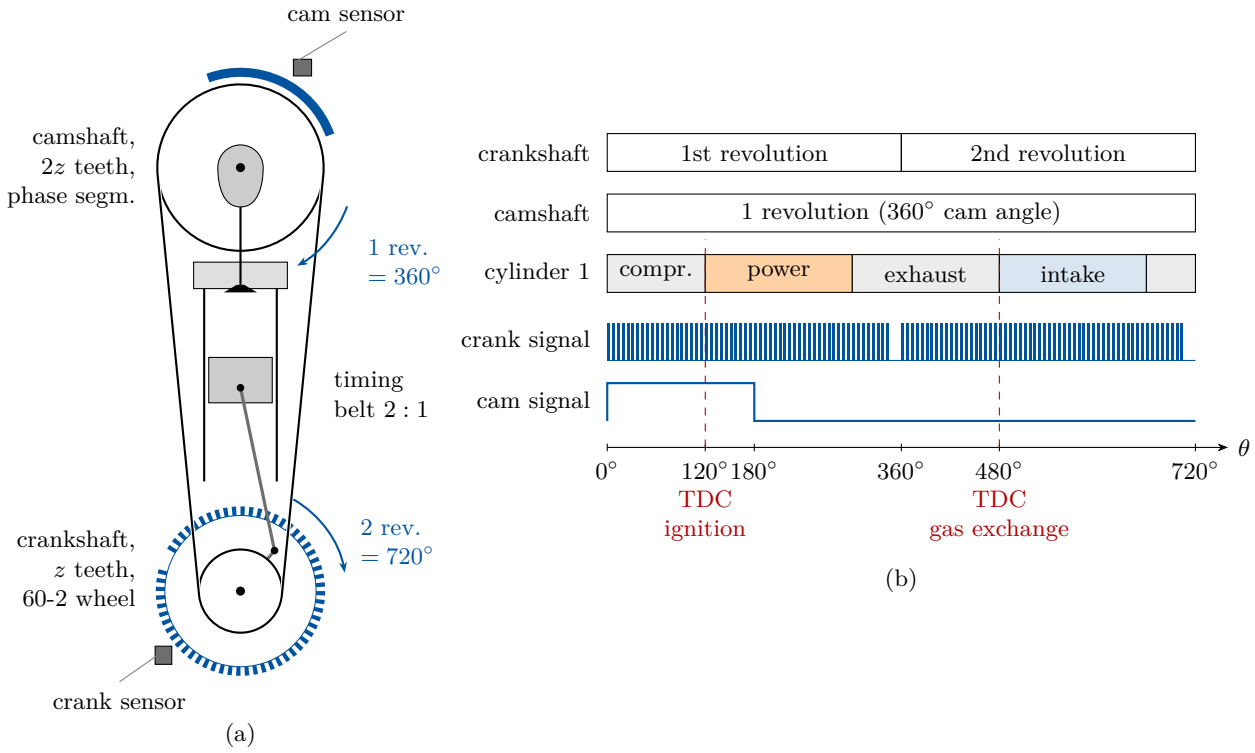


The tooth period T_k is measured from rising edge to rising edge. Because one tooth corresponds to $6° = 1/60$ revolution, the engine speed in rpm is

$$n = \frac{60\,\text{s/min}}{60\,T_k} = \frac{1}{T_k/\text{s}}\ \text{rpm}, \qquad n = \frac{3}{T_{\text{gap}}/\text{s}}\ \text{rpm (gap)}, \qquad n = \frac{60}{T_{\text{rev}}/\text{s}}\ \text{rpm (revolution)}. \tag{1}$$

Background: crankshaft, camshaft and the 720° cycle

A four-stroke engine needs *two* crankshaft revolutions for one working cycle (intake, compression, power, exhaust), but every valve opens only once per cycle. The camshaft is therefore driven by a timing belt (or chain) with a ratio of 2 : 1: its sprocket has twice as many teeth as the crankshaft sprocket, so it turns once while the crankshaft turns twice. The crank angle θ is counted over the full cycle, $0° \le \theta < 720°$, which is exactly one camshaft revolution (360° cam angle). The 60-2 wheel on the crankshaft repeats after 360° crank angle, the single segment of the cam phase wheel only after 720°.



(a) Schematic front view of one cylinder with crank drive and overhead camshaft; the timing belt turns the camshaft once per two crankshaft revolutions. (b) One working cycle of cylinder 1 over the crank angle: the crank wheel pattern and the piston position repeat after 360°, the strokes and the cam signal only after 720°.

Exercise 1.1 Signal classification and time scales

PAPER

a) Classify the five engine signals of Table 1.1 of the script (crank, cam, knock, ion current, lambda) with respect to (i) time: continuous/discrete, (ii) amplitude: analog/binary (digital), (iii) deterministic/stochastic, (iv) periodic/quasi-periodic/aperiodic. Collect the result in a table and justify every entry in one line. Note that the answer may differ for the physical sensor output and for the representation inside the ECU.

b) Discuss the borderline cases: (i) Is the crank signal periodic during an acceleration? Is it periodic in another independent variable? (ii) The knock signal is often called *quasi-periodic*, although a single knock event is a transient. Why? Introduce the notion of a signal whose *statistics* are periodic in the crank angle.

c) Fill in the following table for an engine speed of 3000 rpm: crank revolution period, tooth period and tooth frequency, duration of the gap (rising to rising edge), cam period, cam HIGH time, interval between two combustions (firing frequency), duration of 1° crank angle, time window of a knock onset (5°–25° after TDC), number of 6.5 kHz knock oscillations within one decay time $\tau = 0.8$ ms.

d) Crank and cam: the crankshaft turns 720° per engine cycle, the camshaft 360°. Why is the crank signal alone not sufficient to decide whether the piston of cylinder 1 is in its compression or in its exhaust stroke? After switching on the ignition with the engine cranking at 200 rpm, what is the worst-case time until the ECU knows (i) the crank angle modulo 360°, (ii) the full 720° phase, if the cam level is evaluated at $\theta = 60°$ after the reference mark?

Solution

a) Classification. Go through the four criteria one at a time and ask each time: *what does the sensor deliver*, and *what does the ECU store*?

Step 1: *Time.* Every sensor delivers a voltage at every instant, so all five physical signals are continuous-time (CT). Inside the ECU they become discrete-time (DT) signals, in two different ways: analog signals are *sampled* at fixed times $t = nT_s$ by the ADC; binary signals are *time-stamped* at their edges by a capture timer, so the index is the tooth number k, not a fixed sampling time.

Step 2: *Amplitude.* A Hall sensor switches between two levels. This makes crank and cam binary. Knock, ion current and lambda are analog voltages, and the ADC quantises them (e.g. 10 or 12 bits). The lambda sensor is analog, but its characteristic is so steep that in practice it shows only two values (0.1 V lean, 0.9 V rich).

Step 3: *Predictability.* If the engine speed is known, the tooth wheel and the cam pattern follow exactly from the geometry, so they are deterministic. Knock and noise have random onset, amplitude and phase, so they are stochastic. Combustion varies from cycle to cycle, so the ion current is only partly deterministic.

Step 4: *Periodicity.* Ask after which shift the signal repeats, in time t or in crank angle θ: crank 360°, cam 720°, combustion-related signals $720°/4 = 180°$ (one event per TDC).

This gives the table (physical signal → inside the ECU):

Signal	time	amplitude	predictability	periodicity
crank	CT → event times t_k	binary (Hall)	deterministic up to small speed fluctuations	periodic in θ (360°); in t only at constant speed
cam	CT → level sampled at tooth events	binary	deterministic	periodic in θ (720°)
knock	CT → DT after ADC	analog → quantised	stochastic (noise, random knock onset, amplitude, phase)	quasi-periodic: deterministic parts repeat every 180°, knock is random
ion current	CT → DT	analog	partly deterministic (shape of the peaks), cycle-to-cycle variation	aperiodic per event; event pattern repeats per 180°
lambda	CT → DT (slow, 10–100 Hz)	analog, but practically two-valued (0.1/0.9 V)	slowly varying, disturbed	aperiodic; in the closed loop a limit cycle (quasi-periodic, ≈1 Hz)

Crank and cam are physically continuous-time voltages with two levels. The ECU, however, does not sample them. A capture timer time-stamps their edges, so the information is in the *sequence of edge times* t_k, which is a discrete-time signal indexed by the tooth number. All analog signals become discrete-time and quantised (= digital) after the ADC.

b) Borderline cases. (i) A signal is periodic in t if $x(t) = x(t+T)$ for all t and one fixed $T > 0$. During an acceleration $\omega(t)$ increases, so the tooth period $T_k = 6°/\omega$ gets shorter from tooth to tooth and no single T works: the crank signal is *not* periodic in time. Write it as a function of the crank angle instead, $x(t) = x_c(\theta(t))$ with $\theta(t) = \int_0^t \omega(t')\,\mathrm{d}t'$. The wheel geometry fixes x_c, so $x_c(\theta) = x_c(\theta + 360°)$ holds *exactly*, whatever the speed. This is why many engine algorithms work in the angle domain (Sheet 4).

(ii) Look at a single burst: it is a damped oscillation $A\,\mathrm{e}^{-(t-t_0)/\tau}\sin(2\pi f_1(t-t_0)+\varphi)$ with random t_0, A and φ, and in many cycles there is no burst at all. That is a transient, not a periodic signal. Now look at *averages over many cycles* at a fixed crank angle: the knock *probability*, the

valve impacts, the combustion vibration and the noise level are the same after every TDC. So the mean $\mu(\theta) = \mathrm{E}\{x(\theta)\}$ and the autocorrelation $R(\theta, \theta+\Delta) = \mathrm{E}\{x(\theta)\,x(\theta+\Delta)\}$ satisfy

$$\mu(\theta + 180^\circ) = \mu(\theta), \qquad R(\theta+180^\circ, \theta+180^\circ+\Delta) = R(\theta, \theta+\Delta).$$

A signal whose statistics are periodic like this is called *cyclostationary*. This is exactly what "quasi-periodic" means for the knock signal.

c) Time scales at 3000 rpm**.** Everything follows from two numbers:

$$T_{\mathrm{rev}} = \frac{60}{n} = \frac{60}{3000}\,\mathrm{s} = 20\,\mathrm{ms}, \qquad \omega = \frac{360^\circ}{T_{\mathrm{rev}}} = 18\,000\,^\circ/\mathrm{s}.$$

Each other entry is then an angle divided by ω:

- tooth: $T_z = 6^\circ/\omega = 20\,\mathrm{ms}/60 = 333.3\,\mu\mathrm{s}$, i.e. $f_z = 3\,\mathrm{kHz}$;
- gap: the rising edge of tooth 57 is at 342°, that of tooth 0 at 360°, so $18^\circ/\omega = 3\,T_z = 1\,\mathrm{ms}$;
- cam period = engine cycle: $720^\circ/\omega = 2\,T_{\mathrm{rev}} = 40\,\mathrm{ms}$; cam HIGH for 180°: 10 ms;
- 4 combustions per 720°: one every 180°, i.e. 10 ms, firing frequency 100 Hz = 2× the rotation frequency (2nd order);
- 1°: $1/\omega = 55.6\,\mu\mathrm{s}$, so the knock window 5° … 25° ATDC is 0.278 ms … 1.389 ms after TDC;
- oscillations within τ: $f_1\tau = 6500\,\mathrm{Hz} \cdot 0.8\,\mathrm{ms} = 5.2$.

quantity	formula	value
crank revolution	$60/n$	20 ms (50 Hz)
tooth period / frequency	$T_{\mathrm{rev}}/60$	333 µs / 3 kHz
gap (rising to rising)	$3\,T_z$	1 ms
engine cycle = cam period	$2\,T_{\mathrm{rev}}$	40 ms (25 Hz)
cam HIGH time	180°	10 ms
combustion interval	$720^\circ/4 = 180^\circ$	10 ms (firing frequency 100 Hz = 2nd order)
1° crank angle	$1/\omega$	55.6 µs
knock onset window	5° … 25° ATDC	0.28 ms … 1.39 ms after TDC
knock oscillations in τ	$f_1\tau$	$6.5 \cdot 0.8 = 5.2$

These numbers are easy to check with a few lines of C. Change `n` to get the table for another speed:

```c
double n = 3000.0;                  /* engine speed, rpm      */
double T_rev = 60.0 / n;            /* s per revolution       */
double w = 360.0 / T_rev;           /* crank speed, deg/s     */
printf("tooth       %7.1f us  (%.0f Hz)\n", 1e6 * 6.0 / w, w / 6.0);
printf("gap         %7.3f ms\n", 1e3 * 18.0 / w);
printf("combustion  %7.3f ms  (%.0f Hz)\n", 1e3 * 180.0 / w, w / 180.0);
printf("1 deg       %7.2f us\n", 1e6 / w);
printf("knock onset %7.3f .. %.3f ms ATDC\n", 1e3 * 5.0 / w, 1e3 * 25.0 / w);
```

```
tooth         333.3 us  (3000 Hz)
gap           1.000 ms
combustion   10.000 ms  (100 Hz)
1 deg         55.56 us
knock onset   0.278 .. 1.389 ms ATDC
```

The signals thus span four decades of time scales: knock (100 µs), teeth (300 µs), combustions (10 ms), lambda (1 s).

d) Crank and cam. The crank wheel looks identical in both revolutions of a cycle: at $\theta = 120^\circ$ (compression TDC of cylinder 1, ignition) and at $\theta = 480^\circ$ (gas-exchange TDC of cylinder 1,

valve overlap) the crank signal is the same, and so is the piston position. Only the camshaft, which turns once per cycle, tells the two revolutions apart.
At 200 rpm: $T_{\text{rev}} = 60/200\,\text{s} = 300\,\text{ms}$, so $1° \mathrel{\widehat{=}} 0.833\,\text{ms}$.
(i) The ratio test at the reference edge (360°) needs *three* consecutive rising edges: tooth 56 (336°) and tooth 57 (342°) give T_{prev}, and tooth 57 and the reference edge give T_{gap}. The worst case is to switch on just *after* the edge at 336°. The current gap then cannot be tested (no T_{prev}), and the ECU has to wait for the next reference edge at 720°:

$$t_{360} = (720° - 336°) \cdot 0.833\,\text{ms}/° = 384° \mathrel{\widehat{=}} 320\,\text{ms}.$$

(Best case: switch on just before 336°, sync after $24° = 20\,\text{ms}$.)
(ii) After the reference edge the ECU still has to reach the cam evaluation at 60°:

$$t_{720} = 384° + 60° = 444° \mathrel{\widehat{=}} 370\,\text{ms}.$$

Until then the ECU can use only "wasted spark" (fire cylinders 1 and 4 together at every crank TDC), and it must not inject sequentially.

Exercise 1.2 60-2 crank wheel and engine speed

WOKWI

Open the Wokwi project `code/wokwi/sheet01_crank/` (Arduino Uno, engine simulator chip, logic analyzer, two LEDs, push button). `eng:CRANK` is connected to pin 2 (external interrupt INT0), `eng:CAM` to pin 4. The ISR `crank_isr()` is called at every rising crank edge and time-stamps it. With `TIMEBASE_TIMER1 = 1` the time stamp is the 16-bit Timer1 counter (prescaler 8, 0.5 µs per tick), with 0 it is `micros()` (resolution 4 µs on the Uno).

a) *Preparation (paper).* Derive (1). A timer of resolution Δ quantises the tooth period. Show that the resulting quantisation step of the per-tooth speed is $\Delta n \approx n^2 \Delta$ (with n in rpm and Δ in s) and evaluate it for $\Delta = 4\,\mu\text{s}$ and $0.5\,\mu\text{s}$ at 600, 3000 and 7000 rpm. Compare with the amplitude of the 1 % speed ripple of the model. The gap is detected by the ratio test $T_k > 2\,T_{k-1}$: estimate the angular acceleration (in rpm/s) at which the test would fail (missed gap, or false gap), and compare with realistic engine accelerations ($\lesssim 10\,000$ rpm/s).

b) Complete the ISR: gap detection, tooth counter, per-revolution statistics, TDC LED (pin 12) on for 30° starting at each TDC edge. Which tooth numbers belong to $\theta = 120°$ and 300°? Check with the logic analyzer (D0 = crank, D2 = TDC output of the chip, D3 = your LED) and report the column `teeth` and `sync_err`.

c) Complete the speed computation in `loop()` (per revolution, last tooth, min/max per tooth). At 3000 rpm, compare `rpm_rev` with the spread `rpm_min`…`rpm_max` for both time bases. Press the button to dump 120 tooth periods, copy only this block and plot the per-tooth speed (column `rpm`), e.g. in a spreadsheet or with your CSV plot program of Bonus Exercise 0.B1 d) in JSLinux. Which part of the "jitter" is information, which part is measurement error?

d) Move the speed slider (600…7000 rpm) and repeat c) at 7000 rpm. Why is printing one CSV line per *tooth* not an option (estimate the UART load)?

Solution

a) Speed formulas. Step 1: one tooth is $6° = \frac{1}{60}$ rev. If it takes T_k seconds, the speed is

$$n = \frac{1/60\ \text{rev}}{T_k} = \frac{1}{60\,T_k}\,\frac{\text{rev}}{\text{s}} = \frac{60}{60\,T_k}\,\frac{\text{rev}}{\text{min}} = \frac{1}{T_k/\text{s}}\ \text{rpm}.$$

The gap covers $18° = \frac{3}{60}$ rev and a revolution covers 1 rev, so the same steps give $3/T_{\text{gap}}$ and $60/T_{\text{rev}}$.
Step 2, quantisation: the timer can only measure T_k as an integer multiple of the tick Δ. A

period error of one tick causes a speed error

$$\Delta n \approx \left|\frac{\mathrm{d}n}{\mathrm{d}T}\right| \Delta = \frac{1}{T^2}\,\Delta = n^2\Delta \qquad (n = 1/T \text{ in rpm}, T, \Delta \text{ in s}).$$

Example: at 3000 rpm and $\Delta = 4\,\mu\text{s}$, $\Delta n = 3000^2 \cdot 4 \cdot 10^{-6} = 36$ rpm. Equivalently, the relative error is one tick out of $T_k/\Delta = 333/4 \approx 83$ ticks, i.e. 1.2 %.
Step 3, compare with the ripple: the model speed is $\omega = \omega_0(1 + \varepsilon \sin 2\theta)$ with $\varepsilon = 0.01$, i.e. amplitude $\pm 0.01\,n$:

n / rpm	T_k	Δn ($\Delta = 4\,\mu$s)	Δn ($\Delta = 0.5\,\mu$s)	ripple (±1 %)
600	1667 µs	1.4 rpm	0.2 rpm	±6 rpm
3000	333 µs	36 rpm	4.5 rpm	±30 rpm
7000	143 µs	196 rpm	24.5 rpm	±70 rpm

The relative resolution $\Delta n/n = \Delta/T_k = n\Delta$ gets worse linearly with speed. The quantisation step equals the ripple amplitude when $n^2\Delta = 0.01\,n$, i.e. at $n = 0.01/\Delta$: 2500 rpm for `micros()`, 20 000 rpm for Timer1. Above 2500 rpm the ripple is therefore buried in the `micros()` quantisation. With the 0.5 µs timer it is resolved over the whole speed range. Real ECUs use capture timers with ≤ 100 ns resolution.
Step 4, robustness of the gap test. Let ω_1 be the speed during the tooth before the gap and ω_2 the mean speed across the gap. Then

$$\frac{T_{\text{gap}}}{T_{\text{prev}}} = \frac{18^\circ/\omega_2}{6^\circ/\omega_1} = 3\,\frac{\omega_1}{\omega_2}.$$

Missed gap: the ratio drops to 2 or less if $\omega_2 \geq 1.5\,\omega_1$. The centres of the two intervals are $3^\circ + 9^\circ = 12^\circ$ apart, i.e. $\Delta t \approx 12^\circ/\omega_1$. The speed must therefore rise by $0.5\,\omega_1$ within Δt:

$$\dot\omega \approx \frac{0.5\,\omega_1}{12^\circ/\omega_1} = \frac{\omega_1^2}{24^\circ}.$$

Convert to rpm with $\omega = 6n$ (ω in °/s, n in rpm): $6\dot n = 36n^2/24$, so $\dot n \approx n^2/4$ (rpm/s). At 600 rpm this is $600^2/4 = 90\,000$ rpm/s.
False gap: a normal tooth gives a ratio $T_k/T_{k-1} > 2$ only if the speed halves within one tooth ($\Delta t \approx 6^\circ/\omega$): $\dot\omega \approx -0.5\,\omega^2/6^\circ$, i.e. $\dot n \approx -n^2/2$, which is $-180\,000$ rpm/s at 600 rpm.
Both limits are an order of magnitude beyond real engines ($\lesssim 10\,000$ rpm/s). The offline model confirms the large margin: at 3000 and 7000 rpm the measured gap ratio lies in $2.97 \ldots 3.09$ and the ratio of normal teeth stays below 1.03. So threshold 2 is robust in normal operation (the geometric mean of 1 and 3 would be 1.73). Only the start is critical: when cranking at 200 rpm, the compression strokes cause ±30 % speed fluctuations, and the limit is $\dot n \approx 200^2/4 = 10\,000$ rpm/s. That is why ECUs add plausibility checks there (tooth count 58 between gaps, cam edge position).
b) ISR. The tooth numbers follow from $\theta = 6^\circ k$: $k = 120/6 = 20$ and $k = 300/6 = 50$. The LED stays on for $30^\circ/6^\circ = 5$ teeth. The reference solution (`code/solutions/wokwi/sheet01_crank/sketch.ino`, line numbers as in the file) works in three steps.
Step 1 – time stamp and tooth period. Unsigned subtraction gives the right period even when the counter wraps around. The 16-bit Timer1 wraps every $65536 \cdot 0.5\,\mu\text{s} = 32.8$ ms, so periods up to 32.8 ms are measured correctly. The longest interval is the gap, which limits the method to $n > 3/32.8\,\text{ms} \approx 92$ rpm; that is enough for cranking. The first edge only initialises `t_last`:

```
52 void crank_isr(void)
53 {
54     tick_t now = now_ticks();
```

```
    tick_t T = now - t_last;              /* unsigned: wrap-around safe */
    t_last = now;
    if (!have_edge) { have_edge = true; return; }

    bool gap = (T_prev != 0) && (T > 2 * T_prev);
    T_prev = T;
```

Note `T_prev != 0`: right after the start there is no previous period, so no gap decision is made (this is the extra edge needed in 1.1 d).

Step 2 – gap and tooth counter. At the gap the counter is reset to 0 and the accumulated revolution statistics go to `loop()`. If the previous revolution did not end at tooth 57, an edge was lost or counted twice, and `sync_errors` is incremented. Every other edge increments `tooth` and updates sum, minimum and maximum of T:

```
    if (gap) {
        if (synced && tooth != 57) sync_errors++;
        /* revolution complete: hand over statistics to loop() */
        rev_ticks = rev_acc + T;
        Tmin_rev = Tmin_acc; Tmax_rev = Tmax_acc;
        teeth_rev = teeth_acc + 1;        /* edges incl. this one */
        rev_ready = synced;
        rev_acc = 0; teeth_acc = 0; Tmin_acc = 0xFFFF; Tmax_acc = 0;
        synced = true;
        tooth = 0;
        if (phase >= 0) phase = 360 - phase;   /* next revolution */
    } else {
        if (!synced) return;
        tooth++;
        rev_acc += T;
        teeth_acc++;
        if (T < Tmin_acc) Tmin_acc = T;
        if (T > Tmax_acc) Tmax_acc = T;
        T_tooth_rev = T;
    }
```

Step 3 – TDC LED. Switch it on at teeth 20/50 and off 5 teeth later:

```
    /* crank angle of this edge = phase + 6 * tooth */
    if (tooth == TDC_TOOTH_A || tooth == TDC_TOOTH_B) {
        digitalWrite(TDC_LED, HIGH);
        /* cylinder 1: TDC at 120 deg, only known with cam phase */
        if (phase == 0 && tooth == TDC_TOOTH_A) digitalWrite(CYL1_LED, HIGH);
    }
    if (tooth == TDC_TOOTH_A + LED_TEETH || tooth == TDC_TOOTH_B + LED_TEETH) {
        digitalWrite(TDC_LED, LOW);
        digitalWrite(CYL1_LED, LOW);
    }
```

Expected result: every revolution reports `teeth` = 58 edges (57 normal teeth + the gap edge) and `sync_err` = 0. In the logic analyzer the LED (D3) rises a few microseconds (interrupt latency) after the rising edge of tooth 20 (50) on D0. This is practically together with the 50 µs TDC pulse of the chip on D2, which is emitted exactly at 120°/300°. At 3000 rpm the LED is lit for $5 \cdot 333\,\mu s = 1.67\,ms$ every 10 ms. The LED marks the crank-angle *event*.

c) Speed computation. `loop()` copies the shared variables with interrupts disabled. Without that, the ISR could change a multi-byte value halfway through the read. It then applies (1). The *longest* tooth period gives the *lowest* speed, so `rpm_min` uses `tmax`:

```
        noInterrupts();
        rt = rev_ticks; tmin = Tmin_rev; tmax = Tmax_rev; tl = T_tooth_rev;
        nt = teeth_rev; ph = phase; se = sync_errors; ce = cam_errors;
        rev_ready = false;
        interrupts();
```

```
144         /* rpm = 60 / T_rev[s];  per tooth (6 deg): rpm = 1 / T_tooth[s] */
145         float rpm_rev = 60.0f * TICKS_PER_S / rt;
146         Serial.print(rev++);              Serial.print(',');
147         Serial.print(millis());           Serial.print(',');
148         Serial.print(rpm_rev, 1);         Serial.print(',');
149         Serial.print(TICKS_PER_S / tl, 1); Serial.print(',');
150         Serial.print(TICKS_PER_S / tmax, 1); Serial.print(',');
151         Serial.print(TICKS_PER_S / tmin, 1); Serial.print(',');
```

Step 1 – per-revolution resolution. One revolution lasts 20 ms = 5000 ticks (`micros()`) or 40000 ticks (Timer1). One tick therefore gives $\Delta n = 3000/5000 = 0.6$ rpm, resp. $3000/40000 = 0.075$ rpm.
Step 2 – expected per-tooth values. With the ripple, the tooth period varies by $\pm 1\,\%$, i.e. $T_k = 330.0 \ldots 336.7$ µs. Since `micros()` returns multiples of 4 µs, the measured period is one of 328, 332, 336, 340 µs, which gives the speed levels 3048.8, 3012.0, 2976.2, 2941.2 rpm. These levels are ≈ 36 rpm apart, as predicted in a).
Step 3 – check with the offline model. `code/solutions/wokwi/sheet01_crank/tooth_sim.c` reproduces exactly this measurement over 200 revolutions:

```
$ ./tooth_sim 3000 4
rpm 3000, tick 4.00 us: per-tooth rpm mean 3000.7 std 25.7 min 2941.2 max 3048.8 (11286 teeth)
  per-revolution rpm 2999.40..3000.00
$ ./tooth_sim 3000 0.5
rpm 3000, tick 0.50 us: per-tooth rpm mean 3000.6 std 21.7 min 2967.4 max 3030.3 (11286 teeth)
  per-revolution rpm 2999.85..2999.85
```

(In Wokwi, rare extra outliers of a few µs can occur when the crank interrupt has to wait for the Timer0 or UART interrupt.)

time base	rpm_rev	per-tooth min. . . max	per-tooth std
`micros()`, 4 µs	2999.4. . . 3000.0	2941. . . 3049	25.7 rpm
Timer1, 0.5 µs	2999.85	2967. . . 3030	21.7 rpm

Step 4 – interpretation. The per-revolution speed is slightly *below* 3000. The reason is that the revolution time is $T_{\text{rev}} = \int_0^{360^\circ} \mathrm{d}\theta/\omega$, and the mean of $1/(1+\varepsilon\sin 2\theta)$ is $\approx 1+\varepsilon^2/2$. So $n_{\text{rev}} \approx n(1-\varepsilon^2/2) = 3000 - 0.15$ rpm.
The per-tooth spread splits into two independent parts, which add as variances:

- *Information*: the real ripple, a sine of amplitude 30 rpm with 2 periods per revolution. Its std is $30/\sqrt{2} = 21.2$ rpm. On a real engine these are the torque pulses of the individual combustions, used e.g. for misfire detection.
- *Measurement error*: each period is the difference of two time stamps, and each stamp has a rounding error that is uniform over one tick. The difference therefore has variance $2 \cdot \Delta^2/12 = \Delta^2/6$, i.e. a std of $n^2\Delta/\sqrt{6}$: 14.7 rpm for `micros()`, 1.8 rpm for Timer1.

$$\sigma_{\text{micros}} \approx \sqrt{21.2^2 + 14.7^2} = 25.8\,\text{rpm}, \qquad \sigma_{\text{Timer1}} \approx \sqrt{21.2^2 + 1.8^2} = 21.3\,\text{rpm},$$

in good agreement with 25.7 and 21.7 rpm. With Timer1 almost all of the "jitter" is information. With `micros()`, the measurement error is of the same size as the information. The plotted dump shows a clean sine with 2 periods per 58 teeth for Timer1, and a staircase-like, noisy version for `micros()`. The gap lines (`tooth` = 255) are already converted with $3/T$ in the dump, so they do not show up as outliers.
d) 7000 rpm. Use the same steps with $T_k = 142.9$ µs. `micros()` gives periods 140 . . . 148 µs, i.e. 7142.9 . . . 6756.8 rpm. The predicted std is $\sqrt{(70/\sqrt{2})^2 + (196/\sqrt{6})^2} = \sqrt{49.5^2 + 80.0^2} = 94$ rpm for `micros()` and $\sqrt{49.5^2 + 10.0^2} = 50.5$ rpm for Timer1. The model gives:

```
$ ./tooth_sim 7000 4
rpm 7000, tick 4.00 us: per-tooth rpm mean 7002.1 std 94.2 min 6756.8 max 7142.9 (11286 teeth)
```

```
  per-revolution rpm 6999.53..7002.80
$ ./tooth_sim 7000 0.5
rpm 7000, tick 0.50 us: per-tooth rpm mean 7001.2 std 51.3 min 6920.4 max 7092.2 (11286 teeth)
  per-revolution rpm 6999.53..6999.94
```

With `micros()` the measurement error is now larger than the ripple (±70 rpm). The per-revolution value stays within ±3 rpm (one tick out of 8.57 ms/4 µs = 2143 ticks) resp. 0.5 rpm. When the slider is moved, the speed changes abruptly in the simulator. The gap is still found (ratio 3 vs. 1, see a)) as long as the jump does not fall exactly into the gap.
UART load:

$$\text{edges/s} = \frac{7000}{60} \cdot 58 \approx 6770 \text{ s}^{-1},$$
$$\text{chars/s} \approx 6770 \cdot 20 \approx 135\,000 \text{ s}^{-1} \quad (\text{line } \texttt{1234,57,286,6993.0} + \text{CR/LF}),$$
$$\text{UART capacity} = \frac{115\,200 \text{ bit/s}}{10 \text{ bit/char}} = 11\,520 \text{ chars/s}.$$

Printing one line per tooth would need almost 12× the UART capacity. One line per revolution (≈ 50 chars, 117 rev/s) needs ≈ 5800 chars/s, i.e. half the capacity. Hence statistics per revolution, and a block dump from RAM that is printed after the recording (120 lines of ≈ 16 chars take ≈ 170 ms).

Exercise 1.3 Cam/crank synchronisation

CHALLENGE

Extend your sketch such that it determines the 720° phase from the cam signal (`eng:CAM`, HIGH for $0° \le \theta < 180°$).

a) Why should the cam level not be read at the reference edge ($\theta = 0°$), and which tooth is a good choice instead?

b) Implement `phase` (0° or 360°), toggle it at every gap, and check it at every revolution (`cam_err`). Light the LED on pin 11 only at the compression TDC of cylinder 1 and verify it with the logic analyzer (D1 = cam, D4 = LED).

c) Give the crank angle and the number of the cylinder that fires at each of the four TDC events.

Solution

a) Where to read the cam level. The cam edge at $\theta = 0°$ coincides with the reference edge of the crank signal. In the simulator both are written in the same event. On a real engine the cam edge has a tolerance of several degrees and drifts with cam phasing. Reading the level at the reference edge is therefore a race: the result depends on which edge comes first. The level must be read in the *middle* of a cam state, far from both cam edges (0° and 180°), e.g. at tooth 10 (60° or 420°):

$$\text{HIGH at tooth } 10 \Rightarrow \theta = 60° \text{ (phase } 0°), \qquad \text{LOW at tooth } 10 \Rightarrow \theta = 420° \text{ (phase } 360°).$$

Any tooth whose distance to 0° and 180° is ≥ 10° works, i.e. teeth 2 . . . 28.
b) Implementation. The phase logic consists of two parts. Step 1: *predict*. Every gap starts a new revolution, so the phase toggles between 0 and 360° (inside the gap branch of the ISR):

```
72          if (phase >= 0) phase = 360 - phase;   /* next revolution */
```

Step 2: *measure and check*. At tooth 10 the cam level gives the phase directly. If a prediction exists and the two disagree, `cam_err` is incremented. The measured value always wins, which also does the first synchronisation (`phase` = −1 before):

```
94      /* Ex 1.3: cam level in the middle of a cam state (60 or 420 deg) */
95      if (tooth == CAM_TOOTH) {
```

```
96          int16_t p = digitalRead(CAM_PIN) ? 0 : 360;
97          if (phase >= 0 && p != phase) cam_errors++;
98          phase = p;
99      }
```

Step 3: *use it.* The cylinder-1 LED is switched together with the TDC LED, but only at tooth 20 *and* phase 0°, i.e. at $\theta = 120°$ (line 87 in the listing of Exercise 1.2 b).
Expected result: `cam_err` stays 0 and the printed `phase` alternates 0, 360, 0, … The green LED (D4) is on only every second time the red TDC LED lights up at tooth 20, i.e. once per engine cycle (every 40 ms at 3000 rpm). It always lies inside the cam HIGH phase on D1 (at 120°, cam HIGH for 0 … 180°).
c) Cylinder assignment. The crank angle of an edge is $\theta = \text{phase} + 6°k$. Insert the two TDC teeth and the two phases, then use the TDC positions of the model (cylinders 1, 3, 4, 2 at 120, 300, 480, 660°, firing order 1-3-4-2):

phase	tooth	crank angle	cylinder
0°	20	$0 + 6 \cdot 20 = 120°$	1
0°	50	$0 + 6 \cdot 50 = 300°$	3
360°	20	$360 + 120 = 480°$	4
360°	50	$360 + 300 = 660°$	2

In code this is a small lookup, e.g. `const byte cyl[2][2] = {{1, 3}, {4, 2}};` indexed with `cyl[phase == 360][tooth == 50]`.

Deliverables

1.1: completed classification table and time-scale table. 1.2: sketch, screenshot of the logic analyzer (gap, TDC, LED), table of rpm statistics for both time bases at 3000 and 7000 rpm, plot of a tooth-period dump. 1.3 (optional): extended sketch, logic analyzer screenshot with cam and cylinder-1 LED. Bonus 1.B1 (optional): program, table of statistics and periods, short interpretation.

Bonus: JSLinux (optional)

For interested students: the following exercise analyses all engine signals offline in JSLinux (set up as in Bonus Exercise 0.B1).

Exercise 1.B1 Signal zoo

JSLINUX

The program `code/bonus/jslinux/sheet01/signal_zoo.c` generates 160 ms (four engine cycles) of the crank, cam, knock and ion signals at 3000 rpm, $f_s = 25$ kHz, and 20 s of a lambda sensor signal ($f_s = 200$ Hz) for a slowly oscillating $\lambda(t)$. Additionally, a crank signal during a speed ramp 2000 rpm to 4000 rpm is generated.

a) Complete `stats()`: mean, RMS value, minimum and maximum. Run the program and explain the values of the crank and cam signals analytically. Why is the mean of the knock signal not zero, although the knock sensor is AC coupled?

b) The normalised autocorrelation

$$r[m] = \frac{\hat{R}[m]}{\hat{R}[0]}, \qquad \hat{R}[m] = \frac{1}{N-m} \sum_{n=0}^{N-1-m} (x[n] - \bar{x})(x[n+m] - \bar{x})$$

is close to 1 at lags m where the signal resembles a shifted copy of itself. Complete `autocorr()` and `period_lag()` (first local maximum after the first zero crossing that reaches at least 90 % of the largest value). Report the estimated period and r for all six signals.

c) Interpret the results: Which signals are periodic, which only quasi-periodic? Why does the crank signal show a strong peak already at half a revolution? What happens to the periodicity of the crank signal during the ramp – and which lag is still detected? Why would the $1/N$-normalised (biased) estimate be misleading here?

Solution

a) Statistics. `stats()` needs only one loop over the samples. It accumulates $\sum x$ and $\sum x^2$ and tracks the extremes. Then $\bar{x} = \frac{1}{N}\sum x[n]$ and $x_{\text{rms}} = \sqrt{\frac{1}{N}\sum x[n]^2}$ (reference solution `code/solutions/bonus/jslinux/sheet01/signal_zoo.c`):

```c
static void stats(const char *name, const double *x, int n)
{
    double mean = 0.0, ms = 0.0, mn = x[0], mx = x[0];
    int i;
    for (i = 0; i < n; i++) {
        mean += x[i];
        ms += x[i] * x[i];
        if (x[i] < mn) mn = x[i];
        if (x[i] > mx) mx = x[i];
    }
    mean /= n;
    ms /= n;
    printf("%-14s %9.4f %9.4f %9.4f %9.4f\n", name, mean, sqrt(ms), mn, mx);
}
```

Output:

```
signal              mean       rms       min       max
crank             0.4808    0.6934    0.0000    1.0000
cam               0.2500    0.5000    0.0000    1.0000
knock             0.0433    0.1234   -1.1454    1.5144
ion               0.0993    0.2399   -0.1269    1.0440
lambda            0.5004    0.6195    0.1000    0.9000
```

Binary signals, analytically. Let $x \in \{0, 1\}$ with duty cycle d (fraction of samples that are 1). Then $\bar{x} = d$. Because $x^2 = x$, also $\overline{x^2} = d$, so $x_{\text{rms}} = \sqrt{d}$.

- Crank: 58 teeth with 3° HIGH each per 360°, so $d = 58 \cdot 3/360 = 0.4833$. Measured: 0.4808 and $\sqrt{0.4808} = 0.6934$. The small deviation (0.0025, i.e. about 10 of 4000 samples) is a sampling effect. One sample is $18\,000\,°/\text{s}/25\,\text{kHz} = 0.72°$, so a HIGH phase of 3° lasts only 4.17 sampling intervals and is represented by 4 or 5 samples, depending on where the sampling grid lies. Over the short record these roundings do not average out exactly.
- Cam: $d = 180/720 = 0.25$, RMS $= \sqrt{0.25} = 0.5$ – exact.

Knock mean. The knock signal contains a low-frequency combustion vibration: a positive half sine of amplitude $A = 0.2\,\text{V}$ during 60° after every TDC, i.e. during 60° of every 180°. The mean of a half sine over its own duration is $2A/\pi$, so

$$\bar{x} = A \cdot \frac{2}{\pi} \cdot \frac{60°}{180°} = 0.2 \cdot 0.637 \cdot 0.333 = 0.042\,\text{V},$$

which is the measured mean. (The noise, the valve bursts and the knock oscillations have zero mean.) A real piezo sensor is AC-coupled. The model represents the signal after a high-pass whose cut-off is far below the combustion frequency, so this component survives. The large extremes ($-1.15/+1.51\,\text{V}$) come from knock events.

Lambda RMS. The mean of 0.50 matches a signal that is lean and rich half of the time each. A pure two-level signal (0.1/0.9 V) would have RMS $\sqrt{0.5 \cdot 0.1^2 + 0.5 \cdot 0.9^2} = 0.640$. The measured value is lower (0.6195) because the sensor curve $V = 0.5 + 0.4\tanh((1-\lambda)/0.005)$ with $\lambda = 1 + 0.02\sin\varphi$ spends some time in the transition region: $\overline{V^2} = 0.25 + 0.16\,\overline{\tanh^2(4\sin\varphi)} = 0.25 + 0.16 \cdot 0.836 = 0.384$, so $\sqrt{0.384} = 0.6195$.

b) Autocorrelation and period search. *Small example by hand:* $x = [1,1,0,0,1,1,0,0]$ (period 4), $\bar{x} = 0.5$, deviations ± 0.5. At lag 2 every product is $(+0.5)(-0.5)$, at lag 4 every product is $(\pm 0.5)^2$, so $\hat{R}[0] = 0.25$, $\hat{R}[2] = -0.25$, $\hat{R}[4] = 0.25$, and $r[2] = -1$, $r[4] = 1$. The division by $N - m$ (not N) is what makes $r[4]$ exactly 1, although only 4 products exist at that lag.

`autocorr()` implements the formula literally. First it removes the mean, then it averages $N - m$ products per lag and normalises by $\hat{R}[0]$:

```
48 static void autocorr(const double *x, int n, double *rr, int maxlag)
49 {
50     double mean = 0.0, r0;
51     int i, m;
52     for (i = 0; i < n; i++) mean += x[i];
53     mean /= n;
54     for (m = 0; m <= maxlag; m++) {
55         double s = 0.0;
56         for (i = 0; i + m < n; i++) s += (x[i] - mean) * (x[i + m] - mean);
57         rr[m] = s / (n - m);
58     }
59     r0 = rr[0] > 0.0 ? rr[0] : 1.0;
60     for (m = 0; m <= maxlag; m++) rr[m] /= r0;
61 }
```

`period_lag()` takes three steps. (1) It skips the main lobe around $m = 0$ (every signal resembles itself for small shifts) up to the first zero crossing. (2) It finds the largest value $r_{\max}$ beyond. (3) It returns the *first* local maximum with $r \geq 0.9\, r_{\max}$. Taking the first one avoids picking a multiple of the period, whose peak is almost as high:

```
67 static int period_lag(const double *rr, int maxlag, double *peak)
68 {
69     int m0 = 1, m;
70     double rmax = -1.0;
71     while (m0 <= maxlag && rr[m0] > 0.0) m0++;        /* first zero crossing */
72     for (m = m0; m <= maxlag; m++) if (rr[m] > rmax) rmax = rr[m];
73     for (m = m0 + 1; m < maxlag; m++)
74         if (rr[m] >= rr[m - 1] && rr[m] >= rr[m + 1] && rr[m] >= 0.9 * rmax) {
75             *peak = rr[m];
76             return m;
77         }
78     *peak = 0.0;
79     return 0;
80 }
```

Result:

```
crank 3000   first peak: lag  250 =   10.00 ms r = 0.929 | max: lag  500 r = 0.988
crank ramp   first peak: lag    8 =    0.32 ms r = 0.348 | max: lag    8 r = 0.348
cam          first peak: lag 1000 =   40.00 ms r = 1.000 | max: lag 1000 r = 1.000
knock        first peak: lag  244 =    9.76 ms r = 0.418 | max: lag  500 r = 0.438
ion          first peak: lag  250 =   10.00 ms r = 0.992 | max: lag  250 r = 0.992
lambda       first peak: lag  251 = 1255.00 ms r = 0.981 | max: lag  500 r = 0.982
```

Expected lags at $f_s = 25\,\text{kHz}$: $180° = 10\,\text{ms} = 250$ samples, revolution $= 500$, engine cycle $= 1000$. For lambda at $200\,\text{Hz}$: $1/0.8\,\text{Hz} = 1.25\,\text{s} = 250$ samples.

c) Interpretation.

- *Cam*: exactly periodic, $r = 1.000$ at $40\,\text{ms}$ = one engine cycle.
- *Crank*: the true period is one revolution ($20\,\text{ms}$, $r = 0.988$). It is not exactly 1 because of the speed ripple and because the period is 500.03 samples, not an integer. Why is $r(250)$ almost as high? Shift the wheel by 180°: all teeth fall onto teeth, except that the two missing teeth (58, 59 at 348°, 354°) meet the real teeth 28, 29 at 168°, 174°, and vice versa.

So x and its shifted copy y differ on $2 \cdot 2 \cdot 3^\circ = 12^\circ$ of 360°. With $\overline{(x-y)^2} = 12/360 = 0.033$ and $\overline{xy} - \bar{x}^2 = \sigma^2 - \frac{1}{2}\overline{(x-y)^2}$, where $\sigma^2 = d(1-d) = 0.250$:

$$r(250) \approx 1 - \frac{0.033/2}{0.250} = 0.933 \quad \text{(measured 0.929)}.$$

The gap, which carries the position information, is only a small part of the signal energy. With the 90 % rule the detector therefore reports 180°, a reminder that a period detector must be designed for the feature of interest. The peaks are very narrow: one sample less (0.72°) shifts every tooth edge and gives $r(249) = 0.48$. (Small peaks at multiples of 8.33 samples come from the teeth.)

- *Crank during the ramp*: the tooth period $6^\circ/\omega$ shrinks from 0.5 ms (2000 rpm) to 0.25 ms (4000 rpm), i.e. from 12.5 to 6.25 samples (mean ≈ 8.3). No long-lag peak survives, and only the tooth-scale peak at lag 8 ($r = 0.35$) remains. In time the signal is not periodic. In the angle domain it would be (Sheet 4).
- *Knock*: $r \approx 0.42$ at ≈ 10 ms (180°). Only the deterministic parts (combustion vibration, valve impacts at 8 kHz) repeat. Noise and the random knock bursts (random onset and phase) do not correlate. So the signal is quasi-periodic, i.e. cyclostationary. The lag 244 instead of 250 is a side effect of the 8 kHz valve bursts: their period is only 3.1 samples, so $r[m]$ jumps between 0.30 and 0.44 from lag to lag around $m = 250$. The first local maximum above the threshold is then at 244.
- *Ion current*: $r = 0.992$ at 180°. In the model all cylinders produce the same deterministic ion curve. On a real engine cycle-to-cycle variations make each event an individual (aperiodic) transient.
- *Lambda*: period 1.25 s = 1/0.8 Hz, $r = 0.98$. The switching sensor turns the small λ oscillation into an almost rectangular signal. Noise shifts the switching instants slightly, hence 251 instead of 250.

Biased estimator. With $1/N$ instead of $1/(N-m)$, every $r[m]$ is multiplied by $(N-m)/N$. At lag 1000 of a 4000-sample record this is a factor of $3000/4000 = 0.75$, so the cam signal would show $r = 0.75$ although it is perfectly periodic. The factor also decreases with m, so shorter lags would always be favoured: the crank peak at lag 500 would drop to $0.988 \cdot 0.875 = 0.86$.